Pipelines, Data and Infrastructure

CSC490 Assignment 2

Janis Joplin (1009715051) • Louis Ryan Tan (1009713561)
Maverick Luke (1009714855) • Razan Ahsan Rifandi (1009562108)

January - April 2026

1 Aspirational Datasets

- Repositories and internal onboarding documentation from companies that frequently hire interns and entry-level developers, together with feedback from those developers about how useful the materials were.

  Schema:

  - `repo_link` (string)
  - `docs_link` (string)
  - `intern_ratings` (list of integers 1-5, usefulness of documentation for first contribution)
  - `intern_feedback` (list of strings from surveys or interviews)
  - `junior_ratings` (list of integers 1-5)
  - `junior_feedback` (list of strings)
  - `avg_time_to_first_milestone_hours` (float, how many active hours spent completing the first milestone)
  - `avg_first_milestone_score` (float 1-5, assessed by senior developers)

  This would be the most ideal dataset for our project because it directly connects repository and documentation quality to real onboarding outcomes. Instead of guessing what makes a repository easier to understand, we could observe how long it actually takes new contributors to reach their first meaningful milestone, and how well they perform. That would give us a grounded way to evaluate whether our generated onboarding guide is pointing people in the right direction.

  However, this dataset is extremely difficult to obtain. Internal repositories and documentation are usually private, and onboarding feedback is hard to collect and maintain due to confidentiality issues.

- IDE navigation and reading behavior data from new contributors during their first weeks working on a repository

  Schema:

  - `developer_id` (integer)
  - `session_id` (integer)
  - `repo_link` (string)
  - `file_opened` (string)
  - `timestamp` (datetime)
  - `time_spent_minutes` (float)
  - `navigation_source` (string, e.g. search, go-to-definition, file tree)

  This dataset would help us understand how new contributors actually explore a codebase in practice. Instead of assuming which files are good starting points, we could observe real navigation paths and identify patterns, such as which files are usually opened first or which areas cause people to get stuck. That would let us provide "Smart Starting Points" and structural summaries more grounded in real behavior rather than heuristics.

  However, this data is highly sensitive and usually tied to private IDE concerns. Collecting it requires explicit user consent and cooperation from companies or tooling providers, which makes it unrealistic for our project.

- Historical bug reports and postmortem documents linking issues to specific files and root causes.

  Schema:

- `issue_id` (integer)
- `repo_link` (string)
- `affected_files` (list of strings)
- `root_cause` (string)
- `severity_level` (integer 1-5)
- `resolution_summary` (string)
- `timestamp` (datetime)

This dataset would allow us to better identify fragile areas of a codebase. While commit frequency tells us which files change often, it does not necessarily tell us which ones are likely to break and cause problems across multiple components. By linking real issues to specific files, root causes, and how they were resolved, we could improve our "What Might Break" warnings and highlight parts of the repo that historically have caused major issues.

In reality, postmortem documents and detailed issue reports are almost always internal and confidential. Companies rarely publish it publicly, which makes us think this dataset is aspirational and might not be available.

- Structured code review feedback linked to contributor understanding and follow-up performance.

  Schema:

  - `code_review_id` (integer)
  - `repo_link` (string)
  - `contributor_id` (integer)
  - `pr_title` (string)
  - `pr_description` (string)
  - `pr_changes` (dictionary mapping file name to git diff)
  - `reviewer_id` (integer)
  - `comment` (string)
  - `comment_category` (string, e.g. functionality, style, performance)
  - `contributor_understanding_rating` (integer 1-5)
  - `followup_fix_correct` (boolean, true if they managed to resolve the issue afterward and false otherwise)
  - `timestamp` (datetime)

  This dataset would allow us to better support the Q&A feature of our application. Code review discussions often contain detailed explanations of why a certain implementation is incorrect, what assumptions were misunderstood, and how the logic should be structured instead. By linking review comments with the original pull request context and follow-up fixes, we could learn how experienced engineers explain issues and resolve confusion. These explanation patterns could then inform how our Q&A system responds to similar questions, especially when new contributors misunderstand specific files or design decisions.

  In practice, this kind of structured and outcome-linked review data is rarely collected. Even when pull request comments are public, they rarely contain information about the contributor's understanding of the comment and or records of follow-up changes, which makes this dataset mostly aspirational.

2 Reality Check

We categorize the datasets into two groups: operational datasets used to generate onboarding documents and Q&A responses, and evaluation datasets used to measure the quality and accuracy of the system outputs.

Table 1: Three operational datasets and two evaluation datasets.

Relevant Item	Description	Commentary
GitHub repository snapshot (cloned from user-provided GitHub repo link)	When a user provides a GitHub repo link, our application clones the repo and ingests the full source tree at the latest commit from a specified branch. This includes the full source code, directory structure, commit history, contributor information, and basic repo metadata needed to build an onboarding guide that is grounded in what actually exists in the codebase. For a public repo, we can always get the data, but might require further integration if we want to allow private repos as well.	This serves as the main input to our system. It is what enables us to generate the structural overview (key directories, entry points, important files) and produce evidence-backed outputs, since we can directly point to file paths, specific code locations, and specific code lines in the cloned repo snapshot. In this way, we are treating the repo as a "per-request dataset".
Git commit history and file diffs (extracted from the cloned repository)	From the same cloned repo, we also extract commit metadata and diffs, including commit hash, timestamp, author, files changed, lines added/removed, and commit messages. Similarly, we can always get this from public repos, but might require additional integration for private repos.	This dataset complements ContribNow beyond static code: hotspot detection, high-risk areas (high churn or broad impact), "What might break" warnings, co-change patterns (files frequently modified together), repo evolution summaries (major changes over time), and contributor summaries (what each developer focuses on). It can also provide concrete evidence by linking commit hash and diffs.
GitHub issues and pull requests (extracted from the cloned repository)	From the same cloned repo, we can additionally retrieve publicly available issues and pull requests, including issue and PR titles, descriptions, labels, discussion threads, review comments, and links to related commits. These records provide natural language descriptions of bugs, new features, and design decision within a specific part of the codebase. Similarly, we can always get this from public repos, but might require additional integration for private repos.	This dataset is especially useful for supporting task-based onboarding features and Q&A. Issues and PRs capture how developers in that repo describe problems and changes in their own words, which can be linked back to specific files and commits. When a user wants to work on a task or ask a question, we can pull historically similar issues or PRs and direct them to the relevant files.
Issue and merged PR pairs from some public GitHub repositories	For some well-documented public repos, we collect issues that have been resolved by merged PRs. We treat the issue title and description as a "task prompt", and the files modified in the linked PRs as an approximate ground-truth set of relevant files. This dataset can be obtained through either GitHub API or cloning it ourself, and then extract the issues and commit history. Some examples are FastAPI and Home Assistant.	This dataset is used to evaluate the task-based onboarding and file suggestion features. Given an issue-style task such as fixing a bug or adding a feature, ContribNow should return files that are actually relevant. We can then compare the system's top suggested files with the files that were modified in the real PR. While this is not an accurate ground truth, it provides a scalable and reproducible way to measure retrieval quality. For public repositories, we can collect this directly, but supporting private repositories would require additional integration.

Continued on next page

Relevant Item	Description	Commentary
Repository-specific Q&A prompts with expected evidence files	For a few simple repos, we can generate a list of repo-specific questions, such as "Where are database operations handled?" or "How does the system send messages?". For each question, we manually record a set of expected relevant files or code locations that should be cited as supporting evidence in a generated answer from the Q&A feature. Optionally (we'd like feedback on this), we can also set aside someone who knows the repo inside out to rate the generated answer on some pre-defined metrics. We will start with our team members' repos since we know them. Much more complex repos we might use if needed are Requests (Python HTTP Library) and LLVM Project.	This dataset is used to evaluate the Q&A feature of our system, particularly its evidence quality. Rather than only checking whether a generated answer sounds reasonable, we check whether the response cites relevant files and whether those files actually support the explanation. This helps reduce hallucination and ensures that answers remain traceable to the underlying codebase. Since both the repos and file references are public, and the questions and answers can be manually curated if given enough time, this dataset is realistic to construct within the project scope.

3 Data-processing Pipelines

3.1 Pipeline Overview

ContribNow's data processing pipeline uses a local-first architecture where all processing runs on the user's machine. The pipeline follows a medallion pattern (bronze/silver/gold). The bronze layer contains raw sensitive data and remains on the user's local disk. Starting from the silver layer, all sensitive fields are hashed or stripped. The gold layer is then loaded into a cloud PostgreSQL database that stores only hashed identifiers and aggregated metrics. The database uses deterministic versioning keyed by repo HEAD commit. This enables deduplication when multiple users process the same repository. The pipeline is built in Python, with RDS PostgreSQL as the shared cloud database and IAM policies enforcing least-privilege access.

3.2 Data Schemas

The cloud PostgreSQL database stores only privacy-safe metadata. All sensitive fields (file paths, commit messages, author emails) are hashed or stripped before leaving the user's machine. The database consists of six tables:

tenants

Column	Type	Description
tenant_id	`PK`	Workspace identifier
name	`VARCHAR`	Tenant name
created_at	`TIMESTAMP`	Creation time

repos

Column	Type	Description
repo_id	`PK`	Repository identifier
tenant_id	`FK`	Parent tenant
repo_fingerprint	`VARCHAR`	Hashed repo identifier
canonical_repo_ref_hash	`VARCHAR`	Hash of original ref
Unique: (`tenant_id`, `repo_fingerprint`)		

repo_versions

Column	Type	Description
version_id	`PK`	Version identifier
repo_id	`FK`	Parent repository
head_commit	`VARCHAR`	HEAD commit hash
version_key	`VARCHAR`	Deterministic dedup key
structure_json	`JSONB`	Aggregated metrics
schema_version	`VARCHAR`	Payload schema version
first_seen_at	`TIMESTAMP`	First processed time
last_seen_at	`TIMESTAMP`	Updated on re-runs
Unique: (`version_key`)		

version_hotspots

Column	Type	Description
version_id	`FK`	Parent version
file_hash	`VARCHAR`	Hashed file path
touch_count	`INTEGER`	Commits modifying file
last_touched	`TIMESTAMP`	Last modification time
Unique: (`version_id`, `file_hash`)		

version_files

Column	Type	Description
version_id	`FK`	Parent version
file_path_hash	`VARCHAR`	Hashed file path
content_hash	`VARCHAR`	File content hash (nullable)
Unique: (`version_id`, `file_path_hash`)		

sync_runs

Column	Type	Description
sync_run_id	`PK`	Sync run identifier
tenant_id	`FK`	Parent tenant
user_id	`VARCHAR`	Triggering user
repo_fingerprint	`VARCHAR`	Hashed repo identifier
head_commit	`VARCHAR`	Synced commit
version_key	`VARCHAR`	Deterministic version key
status	`VARCHAR`	success — failed — partial
error_summary	`TEXT`	Error details (nullable)
created_at	`TIMESTAMP`	Sync attempt time

3.3 Pipeline Diagram

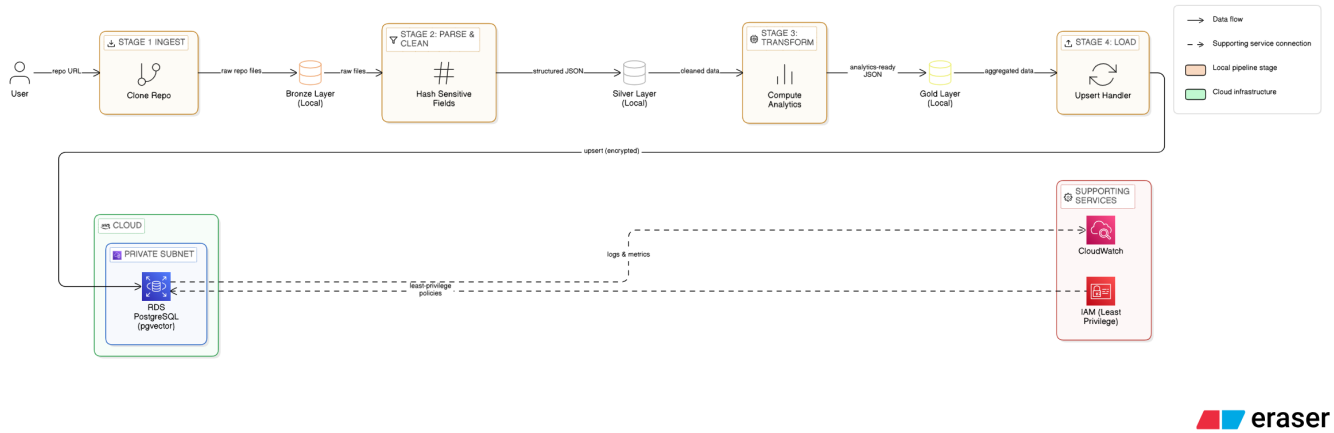

Figure 1: ContribNow Data Processing Pipeline Architecture

3.4 Stage Details

3.4.1 Stage 1: Ingest (Local)

Triggered when a user submits a GitHub repository URL. The pipeline clones the repository using a shallow clone (depth of 500 commits) and stores the raw files on the user's local disk as the bronze layer. This includes full source code, git history, commit messages, and author information. Bronze data is sensitive and never leaves the user's machine.

3.4.2 Stage 2: Parse & Clean (Local → Silver)

Reads from the local bronze layer. Walks the file tree to extract file metadata (language, size) and parses the git log for structured commit data (timestamps, files changed, insertions, deletions). During this stage, sensitive fields are sanitized: file paths are replaced with SHA-256 hashes (using a tenant-scoped salt), commit messages and author emails are stripped, and repository identifiers are replaced with fingerprints. The cleaned, structured output is written locally as the silver layer. Junk directories (`node_modules`, `__pycache__`, `.venv`, `dist`, `build`) and non-source files (`.lock`, binaries, images) are filtered out.

3.4.3 Stage 3: Transform (Local → Gold)

Reads from the local silver layer. Computes hotspot analytics by counting per-file change frequency (using hashed file identifiers) and building a co-change matrix from files that appear in the same commits. Files are assigned risk levels: high (>20 commits), medium (10–20), low (<10). Aggregated structure metrics (total files, directory counts, extension counts) are also computed. The analytics-ready output is written locally as the gold layer.

3.4.4 Stage 4: Load (Gold → PostgreSQL)

Reads from the local gold layer and upserts data into the cloud PostgreSQL database. Uses deterministic version keys (`sha256(tenant_id + repo_fingerprint + head_commit)`) so that repeated runs or multiple users processing the same repo at the same commit produce no duplicate records—only timestamps are updated. Populates six tables: `tenants`, `repos`, `repo_versions`, `version_hotspots`, `version_files`, and `sync_runs`. The `sync_runs` table records each load attempt with status and error information for auditability.

3.5 When Pipelines Run

Implemented:

- **On-demand**: triggered when a user submits a repository URL. The full pipeline (Stages 1-4) runs sequentially, taking approximately 2–10 minutes depending on repository size.

Planned:

- **Scheduled re-index**: a daily job to pull the latest commits for already-indexed repositories and re-run Stages 2–4, keeping hotspot data and co-change patterns up to date.
- **Embedding refresh**: triggered when the embedding model is updated, re-running Stages 3-4 to regenerate vector embeddings.
- **Re-sync on failure**: if a sync run fails or completes partially, retry Stage 4 without re-running the full pipeline. The `sync_runs` table tracks status for this purpose.

3.6 Pipeline Code

The initial pipeline implementation is located in `src/pipeline/`, with separate modules for each stage:

- `src/pipeline/ingest.py`: Stage 1 (clone and store as bronze)
- `src/pipeline/transform.py`: Stages 2 and 3 (parse, clean, compute hotspots)
- `src/pipeline/load.py`: Stage 4 (generate cloud-safe projection)

To run the full pipeline:

```
.venv/bin/python scripts/run_pipeline.py \
    --repo "https://github.com/pallets/markupsafe.git"
```

To include cloud sync to PostgreSQL:

```
.venv/bin/python scripts/run_pipeline.py \
    --repo "https://github.com/pallets/markupsafe.git" \
    --cloud-sync --apply-schema
```

Pipeline artifacts are written locally to `data/raw_<run_id>`, `data/transform_<run_id>`, and `data/output_<run_id>`, corresponding to the bronze, silver, and gold layers respectively.

3.7 Next Steps

The following features were designed but not implemented in the initial version of the pipeline:

- **Migrate processing to ECS Fargate and Lambda**: move pipeline execution from the user's local machine to AWS compute services for scalability and scheduled re-indexing.
- **Embedding generation and RAG-based Q&A**: generate vector embeddings from code chunks and store them in pgvector to enable semantic search, powering a natural language Q&A system over the codebase.
- **PR and issue ingestion via GitHub API**: ingest pull request discussions and issue threads (identified as available in Part 2) to provide richer context for understanding why code changes were made.
- **AST parsing with tree-sitter**: parse source files into abstract syntax trees to enable chunking at function and class boundaries rather than fixed line counts.
- **Convention inference using LLM**: analyze repository patterns and configuration files to infer implicit conventions such as testing practices, naming patterns, and contribution workflows.
- **Onboarding guide generation**: use structured pipeline output as context for an LLM to generate repository-specific onboarding guides for new contributors.
- **Incremental indexing**: track the last-indexed commit per repository and only process new commits on re-runs.

4 Infrastructure as Code Implementation

The data-processing pipeline infrastructure is fully implemented using Terraform as our IaC framework. We chose Terraform for its declarative syntax, modular design, built-in support for multiple environments, and reliable state management. It also supports disaster recovery with `terraform destroy`.

The current pipeline runs locally on the user's machine and uses the `networking` and `rds` modules to provision the cloud PostgreSQL database. The remaining modules (`s3`, `ecs`, `lambda`) are implemented and deployable, provisioned for the planned migration of pipeline processing to AWS as described in Part 3.

The architecture includes:

- VPC networking
- PostgreSQL RDS instance
- IAM roles and security groups
- Environment configuration
- S3 data lake (future)
- ECS Fargate cluster (future)
- Lambda ingestion function (future)
- Application Load Balancer (future)

The terraform project follows this file structure:

```
/modules
    /networking
    /s3
    /rds
    /lambda
    /ecs

/environments
    /dev
    /staging
    /prod
```

The terraform modules we created are as follows:

1. `networking`: Creates a VPC, Subnets (public and private), Internet Gateway, Route Tables, and NAT Gateway. The Internet Gateway is used to allow outbound internet access from public subnets, while the NAT Gateway enables private subnets to access the internet without being publicly exposed. This structure ensures that compute services and DB operate within private subnets for security, while controlled external access is handled through designated public components such as load balancer.

2. `s3`: Creates an S3 bucket that functions as a data lake. We have enabled versioning which helps in case of disaster recovery and accidental deletion. For security, we also configured server-side encryption and blocked public access to prevent accidental public exposure. Furthermore, we have also configured lifecycle to automatically transition old objects. We defined separate data zones: `raw/` for cloned repositories, `processed/` for structured pipeline outputs, and `curated/` for future artifacts such as embeddings or generated documentation.

3. `rds`: Sets up a PostgreSQL database using Amazon RDS. Along with the database itself, it creates a security group and a subnet group in order for the database to run inside private subnets. The database is designed to store structured pipeline data such as repositories, files, commits, and detected hotspots. As it runs in private subnets, it is not directly exposed to the internet, which improves security. It also leaves room to enable the `pgvector` extension later in order to store vector-based features such as embeddings. Sizing can be scaled independently based on the environment (smaller for development and larger for production).

4. `ecs`: Creates an ECS cluster, along with task definitions, an Application Load Balancer (ALB), target groups, security groups, and necessary IAM roles. This module creates a containerized processing environment used for parsing, transformation, and loading stages of the pipeline. The ECS tasks operate within the private subnets for security, and the ALB provides controlled access with load balancing.

5. `lambda`: Provides an AWS Lambda function responsible for triggering the first stage of the pipeline. It creates an IAM execution role, a security group, and a publicly accessible function URL endpoint for receiving repository submissions. This serverless component handles lightweight ingestion logic such as receiving repository URLs and initiating downstream processing.

Three environments are defined with separate `main.tf` files under `/environments`. All environments use `db.t3.micro` to remain within the AWS free tier, but differ in data protection and recovery settings:

Setting	**Dev**	**Staging**	**Prod**
RDS instance size	db.t3.micro	db.t3.micro	db.t3.micro
Deletion protection	Off	Off	On
Backup retention	1 day	3 days	7 days
Skip final snapshot	Yes	Yes	No
Multi-AZ	No	No	No

Instruction on how to use terraform for deploying and destroying infrastructure can be found in the `README.md` file.

We believe that this infrastructure is aligned with the previous parts:

- Part 1 (Aspirational Datasets): Infrastructure supports future embeddings via pgvector in the RDS module, and the S3 module provisions a curated zone for future artifact storage.
- Part 2 (Reality Check): The RDS module stores processed repository metadata. The Lambda and S3 modules are provisioned for future direct GitHub ingestion.
- Part 3 (Pipeline): The current pipeline runs locally and loads data into PostgreSQL via the `rds` module. The `s3`, `ecs`, and `lambda` modules support the planned cloud migration described in Part 3 next steps.

5 Disaster Recovery Demonstration

Our disaster recovery strategy relies on Terraform as the single source of truth for all cloud infrastructure. Since every resource is defined in code, the entire environment can be destroyed and recreated without any manual configuration.

5.1 Destruction Phase

1. Verify the current environment is running: confirm the RDS instance is active, the database contains data, and security groups and VPC are properly configured.
2. Run `terraform destroy -var=db_password='<password>'` to tear down all cloud resources.
3. Verify via the AWS Console that all resources (VPC, subnets, RDS instance, security groups, IAM roles) have been deleted.

5.2 Restoration Phase

1. Run `terraform apply -var=db_password='<password>'` to recreate all infrastructure from code.
2. Run the pipeline with `--cloud-sync --apply-schema` to recreate the database schema and repopulate data:

```
.venv/bin/python scripts/run_pipeline.py \
    --repo "https://github.com/pallets/markupsafe.git" \
    --cloud-sync --apply-schema
```

3. Verify all systems are functional: RDS instance is active, database tables exist and contain data, security groups are correctly configured, and the pipeline can query the database.

5.3 What Gets Restored

Component	Infrastructure	Data
VPC & Networking	Fully recreated	Stateless
RDS PostgreSQL	Instance + config recreated	Repopulated by re-running pipeline
Security Groups	All rules recreated	N/A
IAM Roles	Recreated with same permissions	N/A

5.4 Future Improvements

- Restore from RDS snapshots for exact data recovery (prod environment has `skip_final_snapshot = false`, which preserves a snapshot on destroy)
- Enable automated daily RDS backups with point-in-time recovery
- Store Terraform state in a separate AWS account for additional isolation

Link to video.